

Rheinisch-Westfälische Technische Hochschule Aachen

Bachelor Thesis

Outermost Almost-Sure Termination of Probabilistic Term Rewrite Systems

by

Lucas Ribeiro da Silva

Advisor: Jan-Christoph Kassing

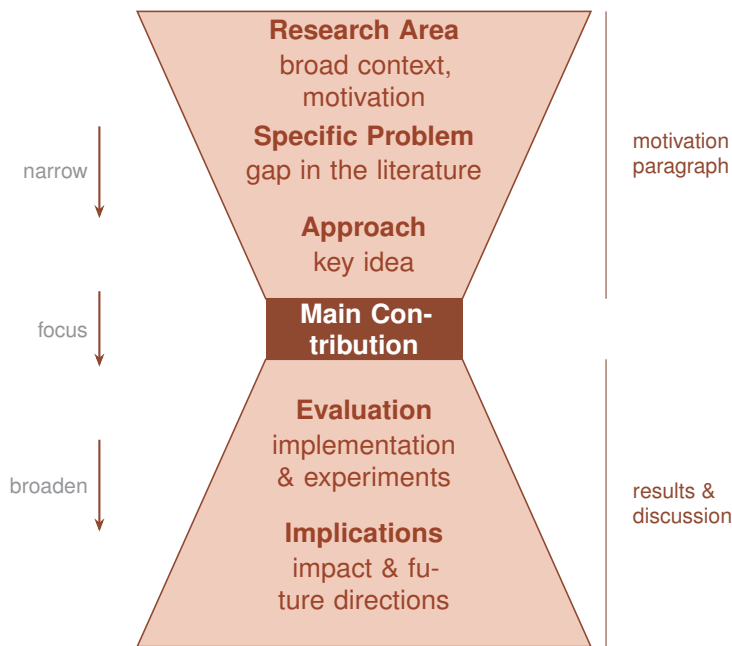
Lehr- und Forschungsgebiet Informatik 2
Programming Languages and Verification

First Examiner:
Prof. Dr. Examiner 1

Second Examiner:
Prof. Dr. Examiner 2

August 11, 2026

Abstract



Hourglass structure of a good abstract. Start broad (research area and problem), funnel down to the specific contribution, then widen again to state results and implications.

Write a self-contained summary of roughly 150–250 words following the hourglass structure above. Cover the following five points in order:

1. **Research area.** Name the field (e.g., automated termination analysis, probabilistic verification, complexity analysis of rewrite systems) and state in one sentence what the field studies and why it matters.
2. **Specific problem.** State precisely which problem this thesis addresses and why it is challenging or not yet fully solved by existing approaches.
3. **Approach.** Describe the key idea of your contribution in one or two sentences. Avoid mathematical notation and abbreviations, and use informal language.
4. **Theoretical results.** Name the main property proved (soundness, completeness, an equivalence result, or a complexity bound) and state it informally.
5. **Implementation and evaluation (if applicable).** Name the tool and the benchmark set (e.g., the Termination Problems Data Base) and give the headline result: how many more examples are solved compared to the baseline and what is the runtime overhead.

-
6. **Implications.** Explain how your results solved the issue or filled the gap you pointed out beforehand.

Acknowledgements

I want to express my most profound appreciation to my supervisor!
Maybe I want to thank someone else as well...

Contents

Abstract	ii
Acknowledgements	iv
Contents	v
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 Related Work	2
1.3 Contribution	3
1.4 Structure of the Thesis	3
2 Preliminaries	5
2.1 Relations and Terms	5
2.2 Term Rewrite Systems and Strategies	7
2.3 Probabilistic Term Rewrite Systems	8
2.4 Probabilistic Derivations	9
2.5 Probabilistic Termination	10
2.6 Transformation-Based Strategy Transfer	12
3 A Probabilistic Outermost-to-Innermost Transformation	13
3.1 The Probabilistic Transformation	13
3.2 Forward Simulation and Soundness	15
3.3 Backward Simulation and Completeness	18
3.3.1 ADP facts used for the extraction	18
3.3.2 Shape of non-converging target computations	19
3.3.3 Projection to source terms	22
3.4 Main Equivalence	24
4 Evaluation	26
4.1 Implementation	26
4.2 Experimental Setup	26
4.3 Results	27
4.4 Discussion	27
5 Conclusion	28
Bibliography	29

Introduction

Term rewriting provides a small but expressive model of computation. A term rewrite system describes a computation by rules that replace matching subterms, and a computation is obtained by applying such rules repeatedly. Termination asks whether this process can continue forever. Besides being a fundamental property in its own right, termination is a prerequisite for using rewrite systems as executable specifications and for reasoning about the result of a computation. It has therefore been studied extensively, both as a mathematical problem and as a target of automated verification; see, for example, [2].

Probabilistic term rewrite systems (PTRSs) extend this model by allowing a rule to choose among several right-hand sides according to a probability distribution. They can represent randomised algorithms and probabilistic program fragments while retaining the syntax and compositional structure of term rewriting. For such systems, classical termination is often unnecessarily strong. A probabilistic computation may admit an infinite run and nevertheless terminate with probability one. This motivates *almost-sure termination* (AST), which requires the probability of eventually reaching a normal form to be one [1]. Importantly, AST is a statement about probability mass rather than about every individual path.

1.1 Motivation and Problem Statement

The order in which redexes are contracted can change termination behaviour. A redex is an occurrence of a subterm that matches the left-hand side of a rule. Under *outermost* rewriting, a redex may be contracted only if it is not strictly contained in another redex. Under *innermost* rewriting, the chosen redex may not contain another redex. These strategies correspond to two opposite evaluation disciplines: outermost rewriting gives priority to the surrounding computation, whereas innermost rewriting evaluates arguments first.

The difference is illustrated by the PTRS in Figure 1.1. Its producer constructs a pair whose first component is chosen probabilistically and whose second component invokes the producer again. Starting with a projection of the producer, outermost evaluation discards the recursive component and returns the first component. Innermost evaluation must instead keep expanding the recursive component before it may apply the projection. The same start term therefore terminates under outermost rewriting but not under innermost rewriting.

A formal account using lifted probabilistic derivations is given in Example 2.3.4.

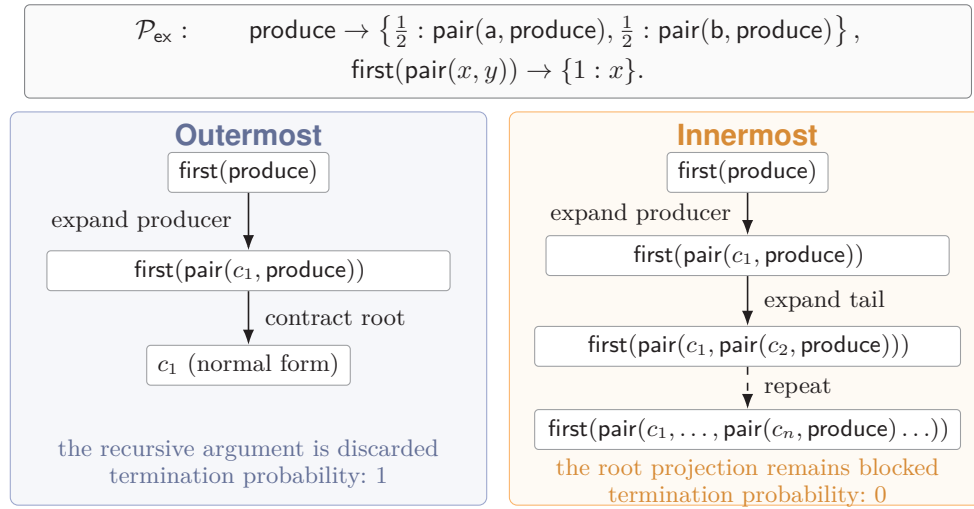


Figure 1.1: Strategy-dependent behaviour from $\text{first}(\text{produce})$. Each c_i is independently chosen as **a** or **b**, each with probability $\frac{1}{2}$.

Automated proof techniques are especially well developed for innermost rewriting. This suggests reducing an outermost termination problem to an innermost one. The central question of this thesis is therefore:

Can outermost almost-sure termination of a PTRS be established by transforming it into a PTRS whose innermost almost-sure termination can be analysed instead?

The intended transformation must do more than reproduce the set of reachable terms. Soundness requires it to preserve the probabilities of the original alternatives and to represent nondeterministic choices between redexes and rules. Completeness additionally requires control over computations beginning with arbitrary terms over the extended signature, including administrative configurations. Such computations must not introduce positive non-termination probability that is absent from the source system. The essential new issue in the probabilistic setting is therefore the preservation of probability mass, rather than merely the existence of infinite reductions.

1.2 Related Work

Transformation-based proofs of strategy-sensitive termination have a history in ordinary term rewriting. Raffelsieper and Zantema transform an outermost termination problem into an unrestricted termination problem. Their principal transformation is sound for outermost ground termination and is effectively constructible as a finite TRS for finite quasi-left-linear systems [5]. Thiemann later introduced a different marker transformation for finite TRSs and proved the exact correspondence

$$\mathcal{R} \text{ is outermost terminating} \iff \mathcal{R}' \text{ is innermost terminating.}$$

This construction is the deterministic starting point for the transformation studied in this thesis [6].

Probabilistic rewriting requires a separate semantic treatment. Avanzini, Dal Lago, and Yamada give a systematic account of PTRSs and probabilistic termination notions [1].

Kassing and Giesl study when innermost results for a PTRS can be transferred to unrestricted probabilistic rewriting, and show that additional syntactic restrictions are needed because duplication can affect probability mass [3]. Their work concerns innermost versus full rewriting; it does not provide the outermost-to-innermost transformation considered here.

The deterministic transformation suggests the construction, but its correctness proof does not transfer automatically to PTRSs. In a deterministic system, one infinite reduction is enough to refute termination. In a probabilistic system, the existence of infinite paths is compatible with AST when the total probability of infinite computations is zero. Consequently, correctness has to be proved for infinite lifted derivations, or equivalently for fully evaluated rewrite sequence trees, while retaining their branch probabilities.

1.3 Contribution

This thesis adapts the marker-based outermost-to-innermost transformation of [6] to probabilistic term rewrite systems. Given a PTRS $\mathcal{P}$, the construction produces a PTRS $\mathcal{P}'$ over an extended signature. Fresh administrative symbols guide an innermost computation to positions that represent permitted outermost steps of $\mathcal{P}$. The rules that move these symbols are deterministic; probabilistic branching occurs only when an original rule is simulated.

The main contributions are:

1. a probabilistic outermost-to-innermost transformation, including the extended signature and its administrative rewrite rules;
2. a simulation result showing that every outermost probabilistic step is represented by a finite, non-empty sequence of innermost steps while preserving the probabilities of all outcomes; and
3. a lifting of the simulation and projection arguments to infinite probabilistic derivations, yielding the sound-and-complete equivalence

$$\text{AST}(\xrightarrow{\circ}_{\mathcal{P}}) \iff \text{AST}(\xrightarrow{i}_{\mathcal{P}'}).$$

Both sides quantify over all terms, including non-ground terms. In particular, the target property concerns every term over the extended signature, not only terms produced by the designated source encoding. The result established here concerns AST; no corresponding claim for PAST or SAST has been made (yet).

1.4 Structure of the Thesis

The remainder of this thesis is organised as follows. Chapter 2 introduces terms, rewrite strategies, finite multi-distributions, probabilistic rewriting, rewrite sequence trees, and the termination notions used throughout the thesis. It also recalls the deterministic transformation result on which the new construction is based. Chapter 3 presents the probabilistic transformation and proves its simulation and correctness properties. Chapter 4 illustrates the construction on representative examples and discusses its applicability and

limitations. Finally, Chapter 5 summarises the results (and maybe outlines possible extensions).

CHAPTER 2

Preliminaries

This chapter fixes the notation and semantic conventions used in the remainder of the thesis. Section 2.1 introduces relations, terms, positions, substitutions, and contexts. Section 2.2 defines term rewrite systems and distinguishes full, outermost, and innermost rewriting. Section 2.3 extends these notions to probabilistic term rewrite systems. The lifted semantics and rewrite sequence trees are presented in Section 2.4, followed by the definitions of AST, PAST, and SAST in Section 2.5. Finally, Section 2.6 recalls the deterministic transformation that forms the starting point of this thesis. Standard background on term rewriting can be found in [2]; the probabilistic conventions follow [1, 3].

2.1 Relations and Terms

We begin with notation for relations. It will be used both for ordinary rewrite relations and for relations between probability distributions. Throughout the thesis, $\mathbb{N} = \{0, 1, 2, \dots\}$.

Definition 2.1.1: Relations, closures, and normal forms

Let A be a set and let $\rightarrow \subseteq A \times A$ be a binary relation. For $n \in \mathbb{N}$, its n -fold composition $\rightarrow^n$ is defined by

$$a \rightarrow^0 b \iff a = b$$

and

$$a \rightarrow^{n+1} b \iff \text{there is a } c \in A \text{ with } a \rightarrow^n c \text{ and } c \rightarrow b.$$

The relations

$$\rightarrow^* = \bigcup_{n \in \mathbb{N}} \rightarrow^n \quad \text{and} \quad \rightarrow^+ = \bigcup_{n \geq 1} \rightarrow^n$$

are the reflexive-transitive and transitive closures of $\rightarrow$, respectively. An element $a \in A$ is a *normal form* if no $b \in A$ satisfies $a \rightarrow b$. The set of all normal forms is denoted by

$$\text{NF}_{\rightarrow} = \{a \in A \mid \nexists b \in A. a \rightarrow b\}.$$

The relation $\rightarrow$ is *terminating*, or *well-founded in the forward direction*, if there is no infinite sequence $a_0 \rightarrow a_1 \rightarrow a_2 \rightarrow \dots$.

A term is a finite tree whose internal nodes are function symbols and whose leaves may additionally be variables.

Definition 2.1.2: Signatures and terms

A *signature* $\Sigma = \uplus_{n \in \mathbb{N}} \Sigma_n$ is a finite set of function symbols with fixed arities; a symbol $f \in \Sigma_n$ has arity n . Let $\mathcal{V}$ be a countably infinite set of variables disjoint from Σ . The set $\mathcal{T}(\Sigma, \mathcal{V})$ of terms over Σ and $\mathcal{V}$ is the least set satisfying:

- every $x \in \mathcal{V}$ is in $\mathcal{T}(\Sigma, \mathcal{V})$; and
- if $f \in \Sigma_n$ and $t_1, \dots, t_n \in \mathcal{T}(\Sigma, \mathcal{V})$, then $f(t_1, \dots, t_n) \in \mathcal{T}(\Sigma, \mathcal{V})$.

Constants are the symbols of arity zero. A term without variables is *ground*; the set of ground terms is denoted by $\mathcal{T}(\Sigma, \emptyset)$. For a term t , $\mathcal{V}(t)$ is the set of variables occurring in t . If $t = f(t_1, \dots, t_n)$, then $\text{root}(t) = f$.

For example, with binary **pair**, unary **first**, and constants **a** and **produce**, the expression **first(pair(a, produce))** is a ground term.

Positions identify nodes in the tree representation of a term. We use the empty sequence ε for the root and concatenate position components with a dot.

Definition 2.1.3: Positions, subterms, and replacement

The set of positions of a term is defined recursively by

$$\text{Pos}(x) = \{\varepsilon\}$$

for variables x , and

$$\text{Pos}(f(t_1, \dots, t_n)) = \{\varepsilon\} \cup \{i.\pi \mid 1 \leq i \leq n, \pi \in \text{Pos}(t_i)\}.$$

For $\pi \in \text{Pos}(t)$, the subterm of t at π is denoted by $t|_\pi$. Replacing this subterm by s yields the term $t[s]_\pi$.

For positions π and τ , we write $\pi \leq \tau$ if π is a prefix of τ , and $\pi < \tau$ if the prefix is proper. In this case, π is *above* τ and τ is *below* π . Positions are *parallel*, written $\pi \parallel \tau$, if neither is a prefix of the other.

For $t = \text{first}(\text{pair}(\mathbf{a}, \text{produce}))$, we have $t|_{1.2} = \text{produce}$ and

$$t[\mathbf{b}]_{1.2} = \text{first}(\text{pair}(\mathbf{a}, \mathbf{b})).$$

Moreover, $\varepsilon < 1 < 1.2$.

Definition 2.1.4: Substitutions and contexts

A *substitution* is a mapping $\sigma : \mathcal{V} \rightarrow \mathcal{T}(\Sigma, \mathcal{V})$ that differs from the identity on only finitely many variables. It extends homomorphically to terms:

$$f(t_1, \dots, t_n)\sigma = f(t_1\sigma, \dots, t_n\sigma).$$

We call $t\sigma$ an *instance* of t .

A *context* C is a term containing exactly one occurrence of a fresh hole symbol $\square$. The term obtained by replacing the hole with t is denoted by $C[t]$.

Thus, a term s occurs as a subterm of t exactly when $t = C[s]$ for a suitable context C .

2.2 Term Rewrite Systems and Strategies

Definition 2.2.1: Term rewrite systems

A *rewrite rule* is a pair $\ell \rightarrow r$ of terms such that $\ell \notin \mathcal{V}$ and $\mathcal{V}(r) \subseteq \mathcal{V}(\ell)$. A *term rewrite system* (TRS) $\mathcal{R}$ is a finite set of rewrite rules.

The root symbols of left-hand sides are the *defined symbols* of $\mathcal{R}$. The remaining symbols of the signature are called *constructors*.

The variable condition prevents a rewrite step from introducing arbitrary new terms through previously unbound variables.

Definition 2.2.2: Rewrite steps and redexes

Let $s, t \in \mathcal{T}(\Sigma, \mathcal{V})$. We write

$$s \xrightarrow{\mathcal{R}, \pi} t$$

if there are a rule $\ell \rightarrow r \in \mathcal{R}$ and a substitution σ such that

$$s|_{\pi} = \ell\sigma \quad \text{and} \quad t = s[r\sigma]_{\pi}.$$

The occurrence $s|_{\pi} = \ell\sigma$ is a *redex*, and π is a *redex position*. We omit π when its value is irrelevant and write $s \xrightarrow{\mathcal{R}} t$.

The next definition restricts which redex positions may be selected. Stating the conditions in terms of redex positions, rather than the eventual target term, is important: a position is disallowed whenever any rule is applicable strictly above or below it.

Definition 2.2.3: Outermost and innermost rewriting

Let $\text{Red}_{\mathcal{R}}(s)$ be the set of redex positions in s . A step $s \xrightarrow{\mathcal{R}, \pi} t$ is

- *outermost*, written $s \xrightarrow{\circ, \pi} t$, if there is no $\tau \in \text{Red}_{\mathcal{R}}(s)$ with $\tau < \pi$; and
- *innermost*, written $s \xrightarrow{\iota, \pi} t$, if there is no $\tau \in \text{Red}_{\mathcal{R}}(s)$ with $\pi < \tau$.

We write $\xrightarrow{\mathcal{R}}$ for unrestricted, or *full*, rewriting and omit positions from $\xrightarrow{\circ, \pi}$ and $\xrightarrow{\iota, \pi}$ when they are not needed.

Equivalently, an innermost step contracts a redex whose proper subterms are all normal forms. Every reducible finite term has both an outermost and an innermost redex. Consequently, full, outermost, and innermost rewriting have the same normal forms, although their infinite derivations can differ.

Definition 2.2.4: Strategy-specific termination

A TRS $\mathcal{R}$ is *outermost terminating* if $\xrightarrow{\circ, \pi}$ is terminating, and it is *innermost terminating* if $\xrightarrow{\iota, \pi}$ is terminating. Full termination is termination of $\xrightarrow{\mathcal{R}}$.

A term is *linear* if every variable occurs at most once. A TRS is left-linear (right-linear) if every left-hand side (right-hand side) is linear, and it is linear if both conditions hold. These properties are not required by the deterministic transformation of [6], but they occur in related strategy-transfer criteria and will be mentioned when those criteria are compared.

2.3 Probabilistic Term Rewrite Systems

Probabilistic rules return finite multi-distributions as in [1, 3]. The prefix “multi” records that two alternatives may lead to syntactically equal terms while still being retained as separate occurrences. This is convenient when individual branches are mapped by a simulation proof.

Definition 2.3.1: Finite multi-distributions

Let A be a non-empty set. A *finite multi-distribution* over A is a finite multiset

$$\mu = \{p_1 : a_1, \dots, p_k : a_k\}$$

with $a_i \in A$, $0 < p_i \leq 1$, and $\sum_{i=1}^k p_i = 1$. The set of all finite multi-distributions over A is denoted by $\text{FDist}(A)$, and $\text{Supp}(\mu)$ denotes the multiset of elements carrying positive weight.

For $0 < q \leq 1$, scaling is defined by

$$q \cdot \mu = \{qp_1 : a_1, \dots, qp_k : a_k\}.$$

If $F : A \rightarrow B$ is a function, its pointwise lifting to multi-distributions is

$$F_{\#}(\mu) = \{p_1 : F(a_1), \dots, p_k : F(a_k)\}.$$

The scaled object $q \cdot \mu$ is a weighted multiset of total weight q , not in general an element of $\text{FDist}(A)$. Such objects are combined with multiset union so that the resulting weights again sum to one.

Definition 2.3.2: Probabilistic term rewrite systems

A *probabilistic rewrite rule* has the form

$$\ell \rightarrow \mu, \quad \mu \in \text{FDist}(\mathcal{T}(\Sigma, \mathcal{V})),$$

where $\ell \notin \mathcal{V}$ and $\mathcal{V}(r) \subseteq \mathcal{V}(\ell)$ for every $r \in \text{Supp}(\mu)$. A *probabilistic term rewrite system* (PTRS) $\mathcal{P}$ is a finite set of probabilistic rewrite rules. We write a rule explicitly as

$$\ell \rightarrow \{p_1 : r_1, \dots, p_k : r_k\}.$$

An ordinary rewrite rule $\ell \rightarrow r$ is identified with the deterministic probabilistic rule $\ell \rightarrow \{1 : r\}$.

PTRSs may be defined more generally as countable sets of rules. This thesis restricts attention to finite PTRSs over finite signatures, as required for the transformation to produce a finite system.

One step still makes a nondeterministic choice of rule and redex position. Once that choice has been made, the right-hand side determines the probabilistic outcomes.

Definition 2.3.3: Probabilistic rewrite steps

Let $\ell \rightarrow \{p_1 : r_1, \dots, p_k : r_k\} \in \mathcal{P}$. If $s|_{\pi} = \ell\sigma$, then s rewrites at π to the multi-distribution

$$\mu = \{p_1 : s[r_1\sigma]_{\pi}, \dots, p_k : s[r_k\sigma]_{\pi}\}.$$

This unrestricted step is written $s \xrightarrow{\mathcal{P}, \pi} \mu$. It is written $s \xrightarrow{\circ}_{\mathcal{P}, \pi} \mu$ if π is an outermost redex position and $s \xrightarrow{\cdot}_{\mathcal{P}, \pi} \mu$ if π is an innermost redex position, using the strategy conditions from Section 2.2.

Normal forms and redex positions depend only on the left-hand sides of $\mathcal{P}$; the probabilities influence the successors, but not where a rule is applicable. We therefore write $\text{Red}_{\mathcal{P}}(s)$ for the positions at which a left-hand side of $\mathcal{P}$ matches a subterm of s .

Example 2.3.4 (Running example). Let $\mathcal{P}_{\text{ex}}$ contain the rules

$$\begin{aligned} \text{produce} &\rightarrow \left\{ \frac{1}{2} : \text{pair}(\text{a}, \text{produce}), \frac{1}{2} : \text{pair}(\text{b}, \text{produce}) \right\}, \\ \text{first}(\text{pair}(x, y)) &\rightarrow \{1 : x\}. \end{aligned}$$

The first probabilistic step from $\text{first}(\text{produce})$ is both outermost and innermost, since the occurrence of produce is the only redex. It yields

$$\left\{ \frac{1}{2} : \text{first}(\text{pair}(\text{a}, \text{produce})), \frac{1}{2} : \text{first}(\text{pair}(\text{b}, \text{produce})) \right\}.$$

In each outcome, the root redex is outermost, while the nested produce redex is innermost. Thus the strategy determines which redex must be selected next.

2.4 Probabilistic Derivations

A term-to-distribution relation describes one probabilistic choice. For a relation $\rightarrow \subseteq A \times \text{FDist}(A)$, we use

$$\text{NF}_{\rightarrow} = \{a \in A \mid \nexists \mu \in \text{FDist}(A). a \rightarrow \mu\}$$

for its normal forms. To follow the probability mass through several steps, it is lifted to a relation between multi-distributions. Normal forms are retained by a self-loop, so all lifted derivations can be viewed as infinite sequences. The following lifting is the standard multidistribution semantics of probabilistic rewriting [1, 3].

Definition 2.4.1: Lifting to multi-distributions

Let $\rightarrow \subseteq A \times \text{FDist}(A)$. Its *lifting* $\Rightarrow \subseteq \text{FDist}(A) \times \text{FDist}(A)$ is defined as follows. For

$$\mu = \{p_1 : a_1, \dots, p_k : a_k\},$$

choose, for every occurrence a_i , a multi-distribution ν_i such that

$$\nu_i = \{1 : a_i\} \quad \text{if } a_i \in \text{NF}_{\rightarrow}, \quad \text{or} \quad a_i \rightarrow \nu_i.$$

Then

$$\mu \Rightarrow \bigoplus_{i=1}^k p_i \cdot \nu_i.$$

Different legal steps may be chosen for different occurrences of the same term. Thus the lifting retains the nondeterminism of the underlying rewrite relation.

For a PTRS $\mathcal{P}$, the liftings of $\xrightarrow{\mathcal{P}}$, $\xrightarrow{\circ}_{\mathcal{P}}$, and $\xrightarrow{\cdot}_{\mathcal{P}}$ are written $\xRightarrow{\mathcal{P}}$, $\xRightarrow{\circ}_{\mathcal{P}}$, and $\xRightarrow{\cdot}_{\mathcal{P}}$, respectively.

In the running example, outermost rewriting gives the lifted derivation

$$\begin{aligned} \{1 : \text{first}(\text{produce})\} &\xRightarrow{\circ}_{\mathcal{P}_{\text{ex}}} \left\{ \frac{1}{2} : \text{first}(\text{pair}(\text{a}, \text{produce})), \frac{1}{2} : \text{first}(\text{pair}(\text{b}, \text{produce})) \right\} \\ &\xRightarrow{\circ}_{\mathcal{P}_{\text{ex}}} \left\{ \frac{1}{2} : \text{a}, \frac{1}{2} : \text{b} \right\}. \end{aligned}$$

2. Preliminaries

Afterwards, both normal forms are retained. Under innermost rewriting, the second lifted step instead expands both nested occurrences of **produce**, and this process continues without producing normal-form probability mass.

Rewrite sequence trees provide an equivalent, and often more convenient, representation. Probabilistic alternatives are represented by branches, whereas the choice of a rule and a redex position is fixed locally at each internal node [3].

Definition 2.4.2: Rewrite sequence trees

Let $\rightarrow \subseteq A \times \text{FDist}(A)$. A *rewrite sequence tree* (RST) $\mathfrak{T}$ for $\rightarrow$ is a rooted, finitely branching tree whose nodes v are labelled by pairs $(p_v : a_v)$ with $0 < p_v \leq 1$. The root has probability 1. If a node v has children $w_1, \dots, w_k$, then

$$a_v \rightarrow \left\{ \frac{p_{w_1}}{p_v} : a_{w_1}, \dots, \frac{p_{w_k}}{p_v} : a_{w_k} \right\}.$$

Leaves are permitted to be reducible. The tree is *fully evaluated* if every leaf is labelled by a normal form. We write $N^{\mathfrak{T}}$ for the nodes and $\text{Leaf}^{\mathfrak{T}}$ for the leaves. The *convergence probability* of the tree is

$$|\mathfrak{T}| = \sum_{v \in \text{Leaf}^{\mathfrak{T}}} p_v.$$

In a partial RST, reducible leaves form an unfinished frontier, so their weight is included in $|\mathfrak{T}|$ even though they have not reached normal form. For a fully evaluated RST, all leaf weight is normal-form weight and $|\mathfrak{T}|$ directly represents the probability of termination.

An infinite branch contributes no leaf probability. Nevertheless, the total probability mass that never reaches a leaf can be zero even if infinite paths exist. This distinction is the reason why ordinary well-foundedness is too strong for probabilistic systems.

2.5 Probabilistic Termination

For a multi-distribution, the probability mass that has already reached a normal form is the following quantity.

Definition 2.5.1: Normal-form probability

Let $\rightarrow \subseteq A \times \text{FDist}(A)$ and $\mu \in \text{FDist}(A)$. Its *normal-form probability* is

$$|\mu|_{\rightarrow} = \sum_{\substack{(p:a) \in \mu \\ a \in \text{NF}_{\rightarrow}}} p.$$

Since the lifting retains normal forms, the sequence $|\mu_0|_{\rightarrow}, |\mu_1|_{\rightarrow}, \dots$ is non-decreasing along a lifted derivation and bounded by one. Its limit therefore exists.

Definition 2.5.2: Almost-sure termination [1]

A term $a \in A$ is *almost-surely terminating* with respect to $\rightarrow$ if every infinite lifted derivation

$$\mu_0 \Rightarrow \mu_1 \Rightarrow \mu_2 \Rightarrow \dots, \quad \mu_0 = \{1 : a\},$$

satisfies

$$\lim_{n \rightarrow \infty} |\mu_n|_{\rightarrow} = 1.$$

The relation $\rightarrow$ is *almost-surely terminating*, written $\text{AST}(\rightarrow)$, if every $a \in A$ is almost-surely terminating.

Equivalently, $\text{AST}(\rightarrow)$ holds if every RST for $\rightarrow$ has convergence probability one; it suffices to consider fully evaluated RSTs.

This termwise formulation is equivalent to quantifying over infinite lifted derivations from arbitrary finite initial multi-distributions, since the finitely many weighted occurrences can be considered separately.

For a PTRS $\mathcal{P}$, *outermost AST* is $\text{AST}(\multimap_{\mathcal{P}})$, and *innermost AST* is $\text{AST}(\dashv_{\mathcal{P}})$. The quantification over every lifted derivation, or every RST, is essential. Probabilistic choices occur as branches inside a tree, while different rule or redex choices yield different trees. AST requires probability-one termination for all resolutions of this nondeterminism.

Example 2.5.3. The single-rule PTRS

$$\text{retry} \rightarrow \{\tfrac{1}{2} : \text{done}, \tfrac{1}{2} : \text{retry}\}$$

has an infinite branch that chooses **retry** every time. After n steps, however, the probability mass still assigned to **retry** is 2^{-n} . Hence the normal-form probability tends to one, and the system is AST even though it is not terminating in the classical sense.

In Example 2.3.4, the term **first(produce)** is almost-surely terminating under outermost rewriting: all probability mass reaches **a** or **b** after two steps. Under innermost rewriting, its normal-form probability remains zero. This is the strategy gap that the transformation in Chapter 3 is designed to bridge.

Two stronger probabilistic termination notions are useful for distinguishing probability-one termination from finite expected runtime.

Definition 2.5.4: PAST and SAST [1, 3]

Let $\vec{\mu} = (\mu_n)_{n \in \mathbb{N}}$ be an infinite lifted derivation. Its *expected derivation length* is

$$\text{edl}(\vec{\mu}) = \sum_{n=0}^{\infty} (1 - |\mu_n|_{\rightarrow}).$$

The relation $\rightarrow$ is *positively almost-surely terminating*, written $\text{PAST}(\rightarrow)$, if $\text{edl}(\vec{\mu}) < \infty$ for every infinite lifted derivation with $\mu_0 = \{1 : a\}$ for some $a \in A$.

For $a \in A$, its *expected derivation height* is

$$\text{edh}_{\rightarrow}(a) = \sup\{\text{edl}(\vec{\mu}) \mid \vec{\mu} = (\mu_n)_{n \in \mathbb{N}} \text{ is a lifted derivation and } \mu_0 = \{1 : a\}\}.$$

The relation is *strongly almost-surely terminating*, written $\text{SAST}(\rightarrow)$, if $\text{edh}_{\rightarrow}(a) < \infty$ for every $a \in A$.

The notions form the strict hierarchy

$$\text{SAST}(\rightarrow) \implies \text{PAST}(\rightarrow) \implies \text{AST}(\rightarrow),$$

in general [1, 3]. This thesis focuses on AST; PAST and SAST are included to make the chosen property precise and to prevent it from being confused with finite expected

derivation length. PAST requires each nondeterministic derivation to have finite expected length; SAST additionally requires a uniform finite bound for all derivations from each fixed start term.

2.6 Transformation-Based Strategy Transfer

The transformation in this thesis is based on the deterministic construction of Thiemann [6]. Its extended signature contains fresh control symbols. Starting from a marked source term, the transformed system first moves a search marker towards a redex. Innermost rewriting permits the marker to descend only when the surrounding source term is not itself a redex; this enforces the outermost restriction. A transformed source rule then performs the actual rewrite step and changes the marker into a return marker. Finally, deterministic administrative rules move that marker back to the root before the next simulated step begins.

Theorem 2.6.1: Deterministic outermost-to-innermost transformation [6]

For every finite TRS $\mathcal{R}$ over a finite signature, the transformation constructs a finite TRS $\mathcal{R}'$ such that

$$\mathcal{R} \text{ is outermost terminating} \iff \mathcal{R}' \text{ is innermost terminating.}$$

Proof idea. — The forward simulation maps every outermost source step to a non-empty finite innermost derivation. For the converse, arbitrary infinite innermost derivations over the extended signature are analysed and a suitable derivation is projected back to an infinite outermost source derivation. The complete proof is given in [6].

An earlier transformation by Raffelsieper and Zantema instead reduces outermost ground termination to unrestricted termination. Its principal correctness result is one-way: termination of the transformed system implies outermost ground termination of the source. For finite quasi-left-linear TRSs, the required anti-matching information and the transformed system can be constructed finitely [5]. This result is useful as a second example of the simulation-based proof pattern, but it must not be confused with the outermost-to-innermost equivalence in Theorem 2.6.1.

These deterministic simulation results motivate the probabilistic construction, but they do not by themselves establish an AST result. The new proof must track probability mass through infinite lifted derivations or fully evaluated RSTs and must represent every nondeterministic outermost source choice. The concrete encoding, the transformed PTRS, and these correctness arguments are introduced in Chapter 3.

A Probabilistic Outermost-to-Innermost Transformation

This chapter presents the main contribution of the thesis: a transformation that reduces outermost almost-sure termination of a probabilistic term rewrite system to innermost almost-sure termination of another probabilistic term rewrite system. The construction follows the marker-based architecture of Thiemann’s deterministic transformation [6]. Its administrative rules are deterministic, while the probabilities of the source rules are retained at the unique point at which a source rewrite step is simulated.

Section 3.1 defines the transformed system and illustrates a complete marker cycle. In Section 3.2, every outermost source step is simulated by a finite sequence of innermost target steps, which yields soundness. The converse direction is established in Section 3.3: annotated dependency pairs are used to isolate the only possible source of non-converging target computations, and a projection then turns such computations into outermost source computations. The two directions are combined in Section 3.4.

3.1 The Probabilistic Transformation

Let $\mathcal{P}$ be a finite PTRS over a finite signature Σ , and write $\text{ar}(f) = n$ whenever $f \in \Sigma_n$. The transformation uses three phases. First, a **reduce**-marker descends from the root towards a chosen outermost redex. Second, the corresponding source rule is applied and the marker changes into **go_up**. Third, the **go_up**-marker returns to the root, where the next simulation cycle begins. The intermediate redex_f -term tests whether the current source subterm is a redex. If it is, innermost rewriting forces the firing rule to be used before a descent rule can be applied. The marker therefore cannot move below a source redex.

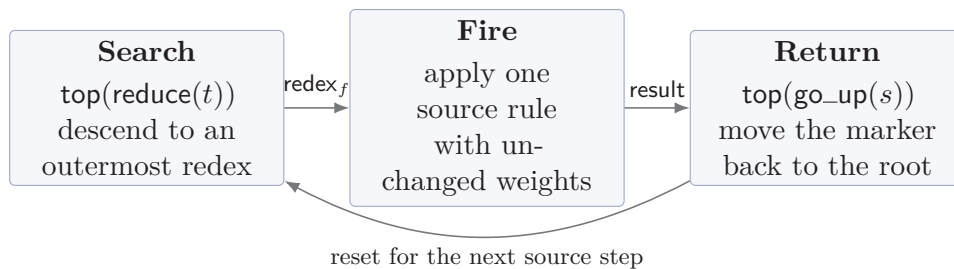


Figure 3.1: The three phases of one simulation cycle.

3. A Probabilistic Outermost-to-Innermost Transformation

We now define the construction formally. All newly introduced symbols are fresh, so none of them occurs in Σ .

Definition 3.1.1: Probabilistic outermost-to-innermost transformation

For every $f \in \Sigma_n$, introduce the unary symbol check_f , the n -ary symbol redex_f , and, for every $1 \leq i \leq n$, the n -ary symbol $\text{in}_{f,i}$. In addition, introduce the unary symbols top , reduce , go_up , and result . The extended signature is

$$\begin{aligned} \Sigma' = \Sigma \uplus \{ & \text{top}, \text{reduce}, \text{go_up}, \text{result} \} \\ & \uplus \{ \text{check}_f, \text{redex}_f \mid f \in \Sigma \} \uplus \{ \text{in}_{f,i} \mid f \in \Sigma_n, 1 \leq i \leq n \}. \end{aligned}$$

The transformed PTRS $\mathcal{P}'$ over Σ' consists of the following rules. In the schemes, $f \in \Sigma_n$, and the variables displayed in an administrative rule are pairwise distinct. For $n = 0$, empty argument lists and their parentheses are omitted.

$$\text{reduce}(f(x_1, \dots, x_n)) \rightarrow \{1 : \text{check}_f(\text{redex}_f(x_1, \dots, x_n))\} \quad (\text{T1})$$

$$\text{check}_f(\text{redex}_f(x_1, \dots, x_n)) \rightarrow \{1 : \text{in}_{f,i}(x_1, \dots, \text{reduce}(x_i), \dots, x_n)\} \quad (\text{T2})$$

$$\text{redex}_f(\ell_1, \dots, \ell_n) \rightarrow \{p_1 : \text{result}(r_1), \dots, p_k : \text{result}(r_k)\} \quad (\text{T3})$$

$$\text{check}_f(\text{result}(x)) \rightarrow \{1 : \text{go_up}(x)\} \quad (\text{T4})$$

$$\text{in}_{f,i}(x_1, \dots, \text{go_up}(x_i), \dots, x_n) \rightarrow \{1 : \text{go_up}(f(x_1, \dots, x_n))\} \quad (\text{T5})$$

$$\text{top}(\text{go_up}(x)) \rightarrow \{1 : \text{top}(\text{reduce}(x))\}. \quad (\text{T6})$$

Here, (T2) and (T5) are included for every $1 \leq i \leq n$. For every source rule

$$f(\ell_1, \dots, \ell_n) \rightarrow \{p_1 : r_1, \dots, p_k : r_k\} \in \mathcal{P},$$

rule (T3) is included with exactly the same probabilities. Finally, (T6) is included once. We write

$$\text{enc}(t) = \text{top}(\text{reduce}(t))$$

for the designated encoding of a source term.

All rules except (T3) have singleton right-hand sides. Thus, administrative rewriting does not create probabilistic branching. If several descent rules or several matching source rules are available, their selection remains nondeterministic, exactly as rule and redex selection is nondeterministic in the source system.

The construction is finite and preserves the variable condition of PTRSs. It introduces

$$4 + 2|\Sigma| + \sum_{f \in \Sigma} \text{ar}(f)$$

fresh symbols and

$$1 + |\mathcal{P}| + 2|\Sigma| + 2 \sum_{f \in \Sigma} \text{ar}(f)$$

rules. In particular, its size is linear in the source rules and in the total arity of the source signature when size is measured by the numbers of symbols and rules.

The following elementary facts will be used repeatedly.

Lemma 3.1.2: Passive source terms and root-redex correspondence

Let $t, t_1, \dots, t_n \in \mathcal{T}(\Sigma, \mathcal{V})$ and $f \in \Sigma_n$.

1. The term t is a normal form with respect to $\mathcal{P}'$.
2. The term $\text{redex}_f(t_1, \dots, t_n)$ is reducible by $\mathcal{P}'$ if and only if $f(t_1, \dots, t_n)$ is a root redex of $\mathcal{P}$.

Proof. Every left-hand side of $\mathcal{P}'$ has a fresh administrative root symbol. Hence no subterm of a term over Σ can be a $\mathcal{P}'$ -redex, which proves the first claim.

The only rules with root redex_f are the instances of (T3). Such a rule matches $\text{redex}_f(t_1, \dots, t_n)$ exactly when the corresponding source left-hand side $f(\ell_1, \dots, \ell_n)$ matches $f(t_1, \dots, t_n)$ at the root. $\square$

Example 3.1.3 (Transformation of the running example). Consider $\mathcal{P}_{\text{ex}}$ from Example 2.3.4. Put

$$u_c = \text{first}(\text{pair}(c, \text{produce})) \quad (c \in \{a, b\})$$

and $\mu = \{\frac{1}{2} : u_a, \frac{1}{2} : u_b\}$. The outermost source step at position 1 is simulated as follows; all displayed arrows are lifted innermost steps of $\mathcal{P}'_{\text{ex}}$:

$$\begin{aligned} \{1 : \text{enc}(\text{first}(\text{produce}))\} &\xrightarrow{i} \{1 : \text{top}(\text{check}_{\text{first}}(\text{redex}_{\text{first}}(\text{produce})))\} \\ &\xrightarrow{i} \{1 : \text{top}(\text{in}_{\text{first},1}(\text{reduce}(\text{produce})))\} \\ &\xrightarrow{i} \{1 : \text{top}(\text{in}_{\text{first},1}(\text{check}_{\text{produce}}(\text{redex}_{\text{produce}})))\} \\ &\xrightarrow{i} \left\{ \begin{array}{l} \frac{1}{2} : \text{top}(\text{in}_{\text{first},1}(\text{check}_{\text{produce}}(\text{result}(\text{pair}(a, \text{produce}))))), \\ \frac{1}{2} : \text{top}(\text{in}_{\text{first},1}(\text{check}_{\text{produce}}(\text{result}(\text{pair}(b, \text{produce})))) \end{array} \right\} \\ &\xrightarrow{i,*} \left\{ \frac{1}{2} : \text{top}(\text{go_up}(u_a)), \frac{1}{2} : \text{top}(\text{go_up}(u_b)) \right\} \\ &\xrightarrow{i} \text{enc}_{\#}(\mu). \end{aligned}$$

The system subscript $\mathcal{P}'_{\text{ex}}$ has been omitted from the arrows to keep the display readable. The first $\text{redex}_{\text{first}}$ -term is not reducible, so innermost rewriting permits descent into its argument. At $\text{redex}_{\text{produce}}$, the transformed probabilistic source rule must fire. The resulting branches have exactly the probabilities of the source step.

3.2 Forward Simulation and Soundness

For a position π , let $|\pi|$ denote its length. Thus $|\varepsilon| = 0$ and $|i.\pi| = 1 + |\pi|$. Let $G : \mathcal{T}(\Sigma', \mathcal{V}) \rightarrow \mathcal{T}(\Sigma', \mathcal{V})$ be defined by $G(t) = \text{go_up}(t)$. Its pointwise lifting $G_{\#}$ to multi-distributions is the lifting from Definition 2.3.1.

3. A Probabilistic Outermost-to-Innermost Transformation

Lemma 3.2.1: Simulation of one outermost step

Let $t \in \mathcal{T}(\Sigma, \mathcal{V})$, and suppose that $t \xrightarrow{\mathcal{P}, \pi} \mu$, where $\mu = \{p_1 : s_1, \dots, p_k : s_k\}$. Then

$$\{1 : \text{reduce}(t)\} \xrightarrow{\mathcal{P}', 3(|\pi|+1)} G_{\#}(\mu).$$

Thus the target derivation is finite and non-empty, and it preserves every outcome and its probability.

Proof. We use induction on $|\pi|$.

First let $\pi = \varepsilon$. There are a source rule

$$\ell \rightarrow \{p_1 : r_1, \dots, p_k : r_k\} \in \mathcal{P}$$

and a substitution σ such that $t = \ell\sigma$ and $s_j = r_j\sigma$ for every j . Write $\ell = f(\ell_1, \dots, \ell_n)$. By construction of $\mathcal{P}'$,

$$\begin{aligned} \{1 : \text{reduce}(t)\} &\xrightarrow{\mathcal{P}'} \{1 : \text{check}_f(\text{redex}_f(\ell_1\sigma, \dots, \ell_n\sigma))\} \\ &\xrightarrow{\mathcal{P}'} \{p_1 : \text{check}_f(\text{result}(r_1\sigma)), \dots, p_k : \text{check}_f(\text{result}(r_k\sigma))\} \\ &\xrightarrow{\mathcal{P}'} \{p_1 : \text{go_up}(r_1\sigma), \dots, p_k : \text{go_up}(r_k\sigma)\} = G_{\#}(\mu). \end{aligned}$$

These steps are innermost by Lemma 3.1.2. In particular, the reducible redex_f -subterm prevents a descent step from being innermost.

Now let $\pi = i.\tau$. Write $t = f(t_1, \dots, t_i, \dots, t_n)$. The contracted source redex lies in t_i at position τ , so there is a multi-distribution $\nu = \{p_1 : u_1, \dots, p_k : u_k\}$ with $t_i \xrightarrow{\mathcal{P}, \tau} \nu$, and

$$\mu = \{p_j : f(t_1, \dots, u_j, \dots, t_n) \mid 1 \leq j \leq k\}.$$

The induction hypothesis gives

$$\{1 : \text{reduce}(t_i)\} \xrightarrow{\mathcal{P}', 3(|\tau|+1)} \{p_1 : \text{go_up}(u_1), \dots, p_k : \text{go_up}(u_k)\}.$$

Since the step at $i.\tau$ is outermost, t is not a root redex of $\mathcal{P}$. By Lemma 3.1.2, $\text{redex}_f(t_1, \dots, t_n)$ is therefore a $\mathcal{P}'$ -normal form. Hence the descent step is innermost, and the induction hypothesis can be used in the surrounding $\text{in}_{f,i}$ -context:

$$\begin{aligned} \{1 : \text{reduce}(t)\} &\xrightarrow{\mathcal{P}'} \{1 : \text{check}_f(\text{redex}_f(t_1, \dots, t_n))\} \\ &\xrightarrow{\mathcal{P}'} \{1 : \text{in}_{f,i}(t_1, \dots, \text{reduce}(t_i), \dots, t_n)\} \\ &\xrightarrow{\mathcal{P}', 3(|\tau|+1)} \{p_j : \text{in}_{f,i}(t_1, \dots, \text{go_up}(u_j), \dots, t_n) \mid 1 \leq j \leq k\} \\ &\xrightarrow{\mathcal{P}'} \{p_j : \text{go_up}(f(t_1, \dots, u_j, \dots, t_n)) \mid 1 \leq j \leq k\} \\ &= G_{\#}(\mu). \end{aligned}$$

The last steps are innermost: the unchanged source arguments and the terms u_j are passive, so every proper subterm of the corresponding $\text{in}_{f,i}$ -redex is a $\mathcal{P}'$ -normal form. The number of steps is $2 + 3(|\tau| + 1) + 1 = 3(|\pi| + 1)$, as required. $\square$

Adding the surrounding top -context and one application of (T6) gives the encoded form of the simulation.

Corollary 3.2.2: Encoded one-step simulation

If $t \xrightarrow{\mathcal{P}, \pi} \mu$, then

$$\{1 : \text{enc}(t)\} \xrightarrow{\mathcal{P}', 3(|\pi|+1)+1} \text{enc}_{\#}(\mu).$$

Proof. Apply Lemma 3.2.1 below **top**, and then apply (T6) to every occurrence in the resulting multi-distribution. $\square$

The same marker discipline also rules out an infinite computation within a single traversal.

Lemma 3.2.3: Finiteness of one marker traversal

Let $t \in \mathcal{T}(\Sigma', \mathcal{V})$ be a $\mathcal{P}'$ -normal form. Starting from $\text{reduce}(t)$, every innermost branch either becomes stuck or reaches a term $\text{go_up}(s)$ after finitely many steps. The subtree up to this first **go_up**-frontier is finite.

Proof. Before a firing rule is used, each descent moves the unique **reduce**-marker into a proper subterm of the finite normal form t . Hence only finitely many descents are possible. A branch then either gets stuck or uses exactly one firing rule. In the latter case, every variable value used by the matching substitution is a subterm of t , hence a normal form. Since source symbols are passive, the firing outcomes are normal forms as well. Therefore (T4) creates **go_up**, and every subsequent return step removes one stored $\text{in}_{f,i}$ -context. Thus the first **go_up**-frontier is reached after finitely many further steps. The branching is finite and occurs only at the single firing step, so the whole first-traversal subtree is finite. $\square$

Source normal forms also produce no administrative divergence.

Lemma 3.2.4: Encoded normal forms terminate administratively

If $t \in \text{NF}_{\mathcal{Q}_{\mathcal{P}}}$, then every innermost derivation from $\text{enc}(t)$ is finite. In particular, every maximal such derivation ends in a $\mathcal{P}'$ -normal form.

Proof. Since all three source strategies have the same normal forms, t contains no $\mathcal{P}$ -redex. Consequently, no transformed firing rule (T3) can be used during the search. Finiteness follows from Lemma 3.2.3. At a variable, or after checking a constant for which no firing rule applies, the marker is stuck. No **go_up**-marker is created, so neither a return rule nor the reset rule can become applicable. $\square$

We can now lift the local simulation to whole rewrite sequence trees. The RST formulation is convenient because different nodes may choose outermost redexes at different depths. Each source edge is replaced independently by its finite simulation fragment.

Theorem 3.2.5: Soundness

If $\mathcal{P}'$ is innermost AST, then $\mathcal{P}$ is outermost AST:

$$\text{AST}(\xrightarrow{\mathcal{P}'}) \implies \text{AST}(\xrightarrow{\mathcal{P}}).$$

Proof. We prove the contrapositive. Suppose that $\mathcal{P}$ is not outermost AST. By Definition 2.5.2, there is a non-converging fully evaluated outermost $\mathcal{P}$ -RST $\mathfrak{T}$, so $|\mathfrak{T}| < 1$.

Replace the term component t of every node label by $\text{enc}(t)$. At each internal node, unfold the finite lifted derivation from Corollary 3.2.2 into an innermost RST fragment. The administrative steps have probability one, and (T3) has exactly the probabilities of the corresponding source rule. The probability of every node is therefore unchanged. At every

source leaf, append any maximal administrative derivation; it is finite and ends in a $\mathcal{P}'$ -normal form by Lemma 3.2.4.

The resulting tree is a fully evaluated innermost $\mathcal{P}'$ -RST $\mathfrak{T}'$ with $|\mathfrak{T}'| = |\mathfrak{T}| < 1$. Hence $\mathcal{P}'$ is not innermost AST. $\square$

3.3 Backward Simulation and Completeness

The converse direction has to cover every term over Σ' , including terms with several administrative symbols that are not produced by enc. A direct inspection of encoded computations is therefore insufficient. Following the role played by ordinary dependency pairs in Thiemann's completeness proof, we use the complete annotated dependency-pair (ADP) framework for innermost AST [4] to extract a non-converging computation of a canonical shape.

The cited ADP framework assumes that the signature contains a constant. If Σ' has none, we adjoin one fresh inert constructor while applying the framework and continue to write Σ' for the extended signature. This does not add a rule or an ADP, and every counterexample RST over the original signature remains a counterexample after the extension. The source projection below maps this target-only constant to a fresh variable. Thus this harmless technical extension does not restrict the main theorem.

3.3.1 ADP facts used for the extraction

We briefly record the ADP notation needed below. This is an application of the existing framework, not part of the transformation itself. For every defined symbol h , let $h^\sharp$ be a fresh annotated copy. If r is a term, $\sharp_{\mathcal{D}}(r)$ denotes the term obtained by annotating all defined symbols in r , and $\flat(r)$ erases every annotation. We use $\flat$ pointwise for multi-distributions, ADPs, and sets of ADPs as well. The canonical ADP associated with $\ell \rightarrow \{p_1 : r_1, \dots, p_k : r_k\}$ is

$$\ell \rightarrow \{p_1 : \sharp_{\mathcal{D}}(r_1), \dots, p_k : \sharp_{\mathcal{D}}(r_k)\}^{\text{true}}.$$

The set of canonical ADPs of a PTRS $\mathcal{Q}$ is denoted by $\text{DP}(\mathcal{Q})$. The flag **true** records that the underlying rule may be used below an annotation while retaining the annotations above the contracted position. The non-probabilistic system $\text{np}(\mathcal{A})$ contains the deannotated support rules of the true-flag ADPs in $\mathcal{A}$. In particular,

$$\text{NF} \xrightarrow{\mathbf{f}}_{\text{np}(\text{DP}(\mathcal{P}'))} = \text{NF} \xrightarrow{\mathbf{i}}_{\mathcal{P}'}.$$

An ADP chain tree is an RST over annotated terms in which distinguished annotated steps track potentially infinite calls; every infinite branch must contain infinitely many such steps. Operationally, a step at an annotation inserts the annotations displayed on the chosen ADP's right-hand side, whereas a step at an unannotated position uses its deannotated right-hand side. A true-flag step below an annotation may retain the annotations above it. These are the only details of the ADP rewrite relation needed below.

We use the following established results from [4, Thm. 4, Lem. 2, Thms. 8, 10, and 12]:

1. the innermost chain criterion states that $\mathcal{Q}$ is innermost AST if and only if $\text{DP}(\mathcal{Q})$ is innermost AST;

2. the Starting Lemma permits a non-converging chain tree to be rooted at an instantiated, root-annotated left-hand side. In the innermost case, this root is an argument normal form: all its proper subterms are normal forms of $\text{np}(\mathcal{A})$;
3. the innermost dependency-graph, usable-terms, and probabilistic reduction-pair processors are sound and complete.

Here soundness means, in particular, that if an input problem is not innermost AST, then at least one generated subproblem is not innermost AST. When a chain tree is turned back into an ordinary RST, annotation erasure preserves every node probability; this is the probability-preserving completeness construction in the proof of the chain criterion.

3.3.2 Shape of non-converging target computations

Write $\mathcal{A}_0 = \text{DP}(\mathcal{P}')$. A root-symbol analysis gives the following sound over-approximation of the relevant part of its innermost dependency graph. The only cyclic strongly connected components are contained in

$$C_{\text{top}} = \{\rho_{\text{top}}\} \quad \text{and} \quad C_{\text{down}} = \{\rho_{\text{red},f}, \rho_{\text{desc},f,i} \mid f \in \Sigma, 1 \leq i \leq \text{ar}(f)\},$$

where the relevant ADPs are represented schematically by

$$\begin{aligned} \rho_{\text{top}} : \quad & \text{top}(\text{go_up}(x)) \rightarrow \{1 : \text{top}^\#(\text{reduce}^\#(x))\}^{\text{true}}, \\ \rho_{\text{red},f} : \quad & \text{reduce}(f(x_1, \dots, x_n)) \rightarrow \{1 : \text{check}_f^\#(\text{redex}_f^\#(x_1, \dots, x_n))\}^{\text{true}}, \\ \rho_{\text{desc},f,i} : \quad & \text{check}_f(\text{redex}_f(x_1, \dots, x_n)) \rightarrow \{1 : \text{in}_{f,i}^\#(x_1, \dots, \text{reduce}^\#(x_i), \dots, x_n)\}^{\text{true}}. \end{aligned}$$

Some annotations displayed here are removed by the usable-terms processor. They are retained in the schematic forms only to show the canonical ADPs before processing.

Indeed, ρ_{top} may lead back to itself or into the downward region. A $\rho_{\text{red},f}$ -ADP may lead to a descent or to an acyclic firing ADP, while a $\rho_{\text{desc},f,i}$ -ADP may lead back to a reduction call or to an acyclic return ADP. The firing and return families do not introduce an annotated call that can lead back to them. These possible edges yield the schematic condensation in Figure 3.2.

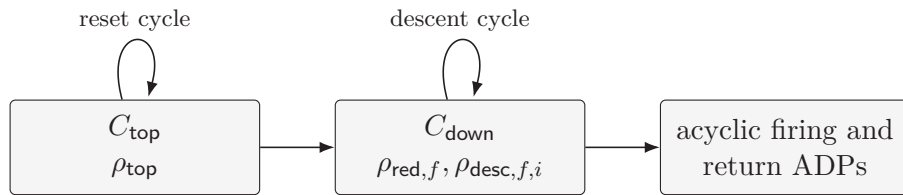


Figure 3.2: Schematic SCC over-approximation for the innermost dependency graph of $\mathcal{A}_0$. Individual downward ADPs that are not cyclic are safely included in C_{down} .

3. A Probabilistic Outermost-to-Innermost Transformation

The downward region cannot itself cause non-convergence. We include the orientation argument because this is the point at which the probabilistic ADP framework is essential. For every cyclic $C \subseteq C_{\text{down}}$, the dependency-graph processor produces $C \cup \flat(\mathcal{A}_0 \setminus C)$. After applying the innermost usable-terms processor, the only right-hand sides that retain annotations have the following forms:

$$\begin{aligned} \text{reduce}(f(x_1, \dots, x_n)) &\rightarrow \{1 : \text{check}_f^\sharp(\text{redex}_f(x_1, \dots, x_n))\}^{\text{true}}, \\ \text{check}_f(\text{redex}_f(x_1, \dots, x_n)) &\rightarrow \{1 : \text{in}_{f,i}(x_1, \dots, \text{reduce}^\sharp(x_i), \dots, x_n)\}^{\text{true}}. \end{aligned}$$

Consider the interpretation A given by the following multilinear polynomials:

$$\begin{aligned} A(f(x_1, \dots, x_n)) &= 1 + x_1 + \dots + x_n & (f \in \Sigma), \\ A(\text{redex}_f(x_1, \dots, x_n)) &= x_1 + \dots + x_n, \\ A(\text{reduce}(x)) &= A(\text{reduce}^\sharp(x)) = 1 + x, \\ A(\text{check}_f(x)) &= A(\text{check}_f^\sharp(x)) = 1 + x, \\ A(\text{in}_{f,i}(x_1, \dots, x_n)) &= x_i, \\ A(\text{result}(x)) &= A(\text{go_up}(x)) = A(\text{top}(x)) = 0. \end{aligned}$$

All remaining annotated symbols, including $\text{in}_{f,i}^\sharp$, $\text{redex}_f^\sharp$, and $\text{top}^\sharp$, are interpreted by zero; the same is done for the fresh inert constant if it was added. These polynomials induce a weakly monotone, $\mathbb{N}$ -collapsible barycentric algebra, as required by the probabilistic reduction-pair processor [4, Sec. 4.4]. All inequalities below are understood pointwise for every valuation of the variables. Let $A_{\text{sum}}^\sharp(r)$ be the sum of the interpretations of all annotated subterms of r , as in the probabilistic reduction-pair processor. For every $\rho_{\text{red},f}$ -ADP we obtain

$$\begin{aligned} A_{\text{sum}}^\sharp(\text{reduce}^\sharp(f(x_1, \dots, x_n))) &= 2 + \sum_{j=1}^n x_j \\ &> 1 + \sum_{j=1}^n x_j \\ &= A_{\text{sum}}^\sharp(\text{check}_f^\sharp(\text{redex}_f(x_1, \dots, x_n))). \end{aligned}$$

For every descent ADP,

$$1 + \sum_{j=1}^n x_j \geq 1 + x_i = A_{\text{sum}}^\sharp(\text{in}_{f,i}(x_1, \dots, \text{reduce}^\sharp(x_i), \dots, x_n)).$$

These inequalities verify weak decrease for the two annotated families; all remaining ADPs have annotation-free right-hand sides and therefore right-hand-side $\sharp$ -sum zero. The strict inequality for $\rho_{\text{red},f}$ is condition (2a) of the processor. Its additional deannotated condition (2b) also holds, since the left-hand side has value $2 + \sum_j x_j$ and its deannotated right-hand side has value $1 + \sum_j x_j$.

It remains to check weak expected decrease of the deannotated support rules, which is condition (3). For (T1) and (T2), the two sides have values $2 + \sum_j x_j \geq 1 + \sum_j x_j$ and $1 + \sum_j x_j \geq 1 + x_i$, respectively. Rules (T4), (T5), and (T6) give $1 \geq 0$, $0 \geq 0$, and $0 \geq 0$. The firing rules are the only non-singleton rules, and they are weakly decreasing in expectation because

$$A(\text{redex}_f(\ell_1, \dots, \ell_n)) \geq 0 = \sum_{j=1}^k p_j A(\text{result}(r_j)).$$

Thus all conditions of the probabilistic reduction-pair processor are satisfied. It removes the annotations from the $\rho_{\text{red},f}$ -ADPs. A second dependency-graph step then returns no cyclic subproblem, because every remaining descent annotation can only lead to one of these now deannotated calls. Consequently, every subproblem represented by C_{down} is innermost AST.

Lemma 3.3.1: Canonical shape of target non-convergence

If $\mathcal{P}'$ is not innermost AST, then there exists a non-converging innermost $\mathcal{P}'$ -RST $\mathfrak{T}$ whose infinite branches repeatedly contain fragments of the form

$$\begin{aligned} (q_j : \text{top}(\text{go_up}(t_j))) &\xrightarrow[\varepsilon]{\mathfrak{T}} (q_j : \text{top}(\text{reduce}(t_j))) \\ &\xrightarrow[\geq \varepsilon]{\mathfrak{T},*} (q_{j+1} : \text{top}(\text{go_up}(t_{j+1}))), \end{aligned} \quad (*)$$

where q_j and q_{j+1} are the cumulative probabilities of the corresponding nodes and every t_j is a $\mathcal{P}'$ -normal form. The arrows in $(*)$ describe a path in $\mathfrak{T}$; they are not lifted multi-distribution arrows. Below every displayed root step, the subtree up to the next $\text{top}(\text{go_up}(\cdot))$ -frontier or an administrative normal form is finite.

Proof. By the innermost chain criterion, $\mathcal{A}_0 = \text{DP}(\mathcal{P}')$ is not innermost AST. Apply the complete innermost dependency-graph processor. All acyclic subproblems are trivially innermost AST, and the preceding reduction-pair argument proves innermost AST of every downward subproblem. Therefore the remaining top subproblem

$$\mathcal{B}_{\text{top}} = C_{\text{top}} \cup b(\mathcal{A}_0 \setminus C_{\text{top}})$$

is not innermost AST. Applying the innermost usable-terms processor removes the non-usable $\text{reduce}^\sharp$ -annotation from the reset ADP, leaving

$$\text{top}(\text{go_up}(x)) \rightarrow \{1 : \text{top}^\sharp(\text{reduce}(x))\}^{\text{true}}.$$

Denote the resulting problem by $\mathcal{B}_{\text{top}}^{\text{UT}}$. Every other ADP in the processed problem has an annotation-free right-hand side. By soundness of the usable-terms processor, this processed problem is still not innermost AST.

The Starting Lemma now gives a non-converging chain tree rooted at an argument-normal instance of a left-hand side with only its root annotated. This left-hand side must belong to ρ_{top} . Otherwise, its first annotated step would remove the only annotation, and no resulting infinite path could contain the infinitely many annotated steps required of a chain tree. Thus the root has the form $\text{top}^\sharp(\text{go_up}(t_1))$. Moreover, the only annotation that can recur is $\text{top}^\sharp$. Consequently, every infinite path repeatedly uses the reset ADP at the root; between two such uses, true-flag steps preserve the outer annotation while rewriting strictly below it.

Erase all annotations. The dependency-graph and usable-terms processors used above only erased annotations and retained every **true**-flag. Consequently,

$$\text{np}(\mathcal{B}_{\text{top}}^{\text{UT}}) = \text{np}(\mathcal{A}_0),$$

whose normal forms coincide with the $\mathcal{P}'$ -normal forms by the chain criterion. Every processed ADP deannotates to the corresponding rule of $\mathcal{P}'$, with the same probabilities.

Thus the probability-preserving erasure construction from the chain criterion preserves innermostness and yields a non-converging innermost $\mathcal{P}'$ -RST. Its recurring distinguished step is precisely the deterministic root step

$$\text{top}(\text{go_up}(t_j)) \xrightarrow{\mathcal{P}', \varepsilon} \{1 : \text{top}(\text{reduce}(t_j))\}.$$

Between two such steps, rewriting occurs strictly below the persistent **top**-symbol, which gives $(*)$.

Finally, the displayed root step is innermost. Hence its proper subterm $\text{go_up}(t_j)$, and therefore t_j , is a $\mathcal{P}'$ -normal form. Applying Lemma 3.2.3 below **top** proves finiteness of every first-return subtree claimed in the statement. $\square$

3.3.3 Projection to source terms

The terms t_j in Lemma 3.3.1 may contain arbitrary symbols from Σ' . We therefore project them to source terms. We harmlessly enlarge the countably infinite variable supply by a disjoint countable set of fresh variables; variable renaming identifies the enlarged supply with the one fixed in Definition 2.1.2.

Definition 3.3.2: Source projection

Let

$$\mathcal{U} = \{u \in \mathcal{T}(\Sigma', \mathcal{V}) \setminus \mathcal{V} \mid \text{root}(u) \in \Sigma' \setminus \Sigma\}.$$

Choose a set $Z = \{z_u \mid u \in \mathcal{U}\}$, disjoint from $\mathcal{V}$, whose elements are pairwise distinct, and put $\widehat{\mathcal{V}} = \mathcal{V} \uplus Z$. Define

$$\text{src} : \mathcal{T}(\Sigma', \mathcal{V}) \longrightarrow \mathcal{T}(\Sigma, \widehat{\mathcal{V}})$$

by

$$\begin{aligned} \text{src}(x) &= x & (x \in \mathcal{V}), \\ \text{src}(f(t_1, \dots, t_n)) &= f(\text{src}(t_1), \dots, \text{src}(t_n)) & (f \in \Sigma), \\ \text{src}(u) &= z_u & (\text{root}(u) \in \Sigma' \setminus \Sigma). \end{aligned}$$

For a target substitution $\delta : \mathcal{V} \rightarrow \mathcal{T}(\Sigma', \mathcal{V})$, define the source substitution $\text{src}(\delta)$ on $\widehat{\mathcal{V}}$ by

$$x \text{ src}(\delta) = \text{src}(x\delta) \quad (x \in \mathcal{V}), \quad z \text{ src}(\delta) = z \quad (z \in Z).$$

The term map is lifted pointwise to multi-distributions as $\text{src}_\#$.

Fresh variables deliberately hide the internal structure of administrative subterms. Choosing a different matching variable for every such term makes src injective, which is needed to reflect source matching.

Lemma 3.3.3: Projection and matching

Let $\ell \in \mathcal{T}(\Sigma, \mathcal{V})$ and $t \in \mathcal{T}(\Sigma', \mathcal{V})$.

1. For every target substitution δ , $\text{src}(\ell\delta) = \ell \text{ src}(\delta)$.
2. If σ is a substitution over $\widehat{\mathcal{V}}$ and $\text{src}(t) = \ell\sigma$, then there is a target substitution δ such that $t = \ell\delta$.

Proof. The first claim follows by structural induction on ℓ .

For the second claim, observe first that src is injective: source-rooted terms retain their root and are handled homomorphically, whereas every administrative-rooted term is mapped to its own fresh variable. Every value $x\sigma$ occurring in the equality $\text{src}(t) = \ell\sigma$ is therefore the projection of a unique corresponding subterm of t . Assign that subterm to $x\delta$. Repeated occurrences of x give the same subterm by injectivity. A structural induction on ℓ now yields $t = \ell\delta$. $\square$

We can now reverse a complete marker traversal. The statement is multi-distribution-valued: this is where the probabilities of the transformed firing rule are transferred back to the source system.

Lemma 3.3.4: Extraction of one outermost step

Let $t \in \mathcal{T}(\Sigma', \mathcal{V})$ be a $\mathcal{P}'$ -normal form. If

$$\{1 : \text{reduce}(t)\} \xrightarrow{\mathcal{P}'}^+ G_{\#}(\mu),$$

then

$$\text{src}(t) \xrightarrow{\mathcal{P}} \text{src}_{\#}(\mu),$$

and every term in $\text{Supp}(\mu)$ is a $\mathcal{P}'$ -normal form.

Proof. Normal-form retention may add self-loops after the traversal has finished; we discard such a suffix and consider the first multi-distribution whose terms all have root **go_up**. We use induction on the number of applications of the descent rule (T2). Since the derivation reaches a term with root **go_up**, its first step must use (T1). Thus $t = f(t_1, \dots, t_n)$ for some $f \in \Sigma_n$, and all t_i are $\mathcal{P}'$ -normal forms.

First suppose that the transformed firing rule is used next. Then there are a source rule

$$f(\ell_1, \dots, \ell_n) \rightarrow \{p_1 : r_1, \dots, p_k : r_k\} \in \mathcal{P}$$

and a substitution δ with $t_i = \ell_i\delta$. The only possible continuation to a **go_up**-rooted multi-distribution uses (T4), so $\mu = \{p_1 : r_1\delta, \dots, p_k : r_k\delta\}$. By Lemma 3.3.3,

$$\text{src}(t) = f(\ell_1, \dots, \ell_n) \text{src}(\delta) \xrightarrow{\mathcal{P}, \varepsilon} \{p_j : r_j \text{src}(\delta) \mid 1 \leq j \leq k\} = \text{src}_{\#}(\mu).$$

Each $r_j\delta$ is a $\mathcal{P}'$ -normal form: its source symbols are passive, and every substitution value that occurs in r_j also occurs in the left-hand side and is therefore a subterm of the normal form t .

Otherwise, the derivation uses (T2) for some argument i . It first produces

$$\text{in}_{f,i}(t_1, \dots, \text{reduce}(t_i), \dots, t_n).$$

Before the outer $\text{in}_{f,i}$ -rule can return, the inner marker must complete a shorter traversal

$$\{1 : \text{reduce}(t_i)\} \xrightarrow{\mathcal{P}'}^+ G_{\#}(\nu).$$

The induction hypothesis yields $\text{src}(t_i) \xrightarrow{\mathcal{P}} \text{src}_{\#}(\nu)$, and all outcomes of ν are normal forms of $\mathcal{P}'$. The final return rule puts these outcomes back into the i -th argument of f . More explicitly, if $\nu = \{q_h : u_h \mid 1 \leq h \leq m\}$, then

$$\mu = \{q_h : f(t_1, \dots, u_h, \dots, t_n) \mid 1 \leq h \leq m\},$$

and pointwise projection gives the corresponding contextual lifting of the source step.

It remains to show that the resulting contextual source step is outermost. If $\text{src}(t)$ were a root redex of $\mathcal{P}$, the matching reflection in Lemma 3.3.3 would imply that t matches the corresponding source left-hand side. Then $\text{redex}_f(t_1, \dots, t_n)$ would be reducible below check_f , so the descent rule could not have been applied innermost. Thus the root is not a source redex. The induction hypothesis already excludes every redex at an ancestor position inside t_i , so no source redex is above the contextual step. It is therefore outermost. Its outcomes are normal forms of $\mathcal{P}'$, since the unchanged arguments and all outcomes of ν are normal and f is passive in $\mathcal{P}'$. $\square$

Theorem 3.3.5: Completeness

If $\mathcal{P}$ is outermost AST, then $\mathcal{P}'$ is innermost AST:

$$\text{AST}(\xrightarrow{\mathcal{O}}_{\mathcal{P}}) \implies \text{AST}(\xrightarrow{\mathcal{I}}_{\mathcal{P}'}).$$

Proof. Again we prove the contrapositive. Suppose that $\mathcal{P}'$ is not innermost AST. By Lemma 3.3.1, there is a non-converging innermost $\mathcal{P}'$ -RST whose infinite branches repeatedly have the form $(*)$, with every t_j in normal form.

First normalize every finite fragment that ends partway through a marker traversal: extend it only until it reaches the next $\text{top}(\text{go_up}(\cdot))$ -frontier or an administrative normal form. This is always a finite expansion by Lemma 3.2.3. Replacing a frontier leaf by such a finite subtree preserves its total weight, so this normalization does not change the convergence probability.

Now consider a macro-node $(q : \text{top}(\text{go_up}(t)))$. Its reset child is $(q : \text{top}(\text{reduce}(t)))$. If the subsequent administrative search gets stuck, leave the corresponding source macro-node unexpanded. Otherwise, collect all nodes at the next $\text{top}(\text{go_up}(\cdot))$ -frontier. Dividing their weights by q gives a multi-distribution $G_{\#}(\mu)$, and the finite first-return subtree represents

$$\{1 : \text{reduce}(t)\} \xrightarrow{\mathcal{I}}_{\mathcal{P}'}^+ G_{\#}(\mu).$$

This works at the multi-distribution level, rather than for just one path: before the firing rule there is no probabilistic branching, and after it all branches perform only the deterministic return rules.

Contract this whole first-return subtree to one edge, remove the outer administrative context, and label the macro-node by $\text{src}(t)$. At every macro-node with children, Lemma 3.3.4 proves that the new edge is one outermost $\mathcal{P}$ -step and that the relative child weights are exactly its source probabilities. Hence the contracted object is an outermost $\mathcal{P}$ -RST.

Contraction changes neither the probability of a macro-node nor the weight of any leaf: administrative edges have weight one, and every firing frontier is grouped with all of its probabilistic children. Thus the source RST has the same convergence probability, which is smaller than one. Its variables lie in the countably infinite set $\hat{\mathcal{V}}$; applying a bijective variable renaming identifies this with the supply $\mathcal{V}$ fixed in the preliminaries and preserves all rewrite steps. Therefore $\mathcal{P}$ is not outermost AST. $\square$

3.4 Main Equivalence

Combining Theorems 3.2.5 and 3.3.5 yields the desired result.

Theorem 3.4.1: Soundness and completeness of the transformation

Let $\mathcal{P}$ be a finite PTRS over a finite signature Σ , and let $\mathcal{P}'$ be the PTRS from Definition 3.1.1. Then

$$\text{AST}(\xrightarrow{\mathcal{P}}) \iff \text{AST}(\xrightarrow{\mathcal{P}'}).$$

The right-hand side quantifies over all terms over the extended signature Σ' , not only terms of the form $\text{enc}(t)$.

Proof. The implication from right to left is Theorem 3.2.5; the implication from left to right is Theorem 3.3.5. $\square$

The theorem concerns AST. The simulation preserves the probability of every source outcome, but it inserts a number of administrative steps that depends on the depth of the selected redex. Therefore the result does not by itself imply an analogous equivalence for PAST or SAST, both of which additionally constrain expected derivation lengths.

CHAPTER 4

Evaluation

This chapter demonstrates that the approach from Chapter 3 works in practice.

A good evaluation chapter:

- Describes clearly what was implemented and where.
- Explains the experimental setup with enough detail to be reproducible.
- Presents results honestly, including cases where the technique does not help.
- Interprets the numbers and connects them back to the theory.

4.1 Implementation

The implementation section should cover:

1. The name of the tool extended and a brief description of its overall architecture (e.g., the pipeline from input parsing to proof output). and any external solvers used.
2. Which module(s) you added or changed and how they fit into the existing architecture.

4.2 Experimental Setup

The experimental setup should specify:

1. **Benchmark set.** Which set was used (e.g., TPDB, TermComp, SV-COMP, or a custom set)? How many examples total? Are there relevant subsets by category, strategy, or problem type?
2. **Configurations compared.** At minimum include the unmodified baseline and the tool with your extension. List any competing tools with a brief description.
3. **Hardware and time limits.** CPU model, amount of RAM, operating system, and per-example wall-clock timeout.

4.3 Results

The results section should:

1. Present results in a table (see Table 4.1).
2. Summarize the headline finding directly below the table, citing concrete numbers (additional YES answers, percentage improvement, overhead in seconds).
3. Point out interesting subsets or individual examples where the technique is especially effective and where the technique is not effective.
4. Note any surprising results to be discussed in Section 4.4.

Table 4.1 summarizes the results of our experiments.

Table 4.1: Experimental results on [*benchmark set name*]. *YES/NO* denotes the number of examples for which [*the property / its negation*] was proved. *MAYBE* means the tool returned no answer. *TO* means the timeout was exceeded. *Time* is the total wall-clock time in seconds.

Configuration	YES	NO	MAYBE	Avg. Time (s)
<i>Tool without extension</i>	–	–	–	–
<i>Tool with extension</i>	–	–	–	–

4.4 Discussion

The discussion should address the following questions:

1. Do the results match theoretical expectations from Chapter 3? If not, explain why.
2. On which types of examples does the technique help most? Identify structural properties of the benchmark examples that explain the gain.
3. Are there cases where the technique does not help or is slower? Explain the reason and relate it to the theoretical properties.
4. What are the main limitations and how could they be overcome? (This can bridge to the future work discussion in Chapter 5.)

CHAPTER 5

Conclusion

The conclusion summarizes the thesis and discusses future work. Do not introduce any new material here. A reader who reads only the introduction and the conclusion should get a coherent picture of what was done and what remains open. It should follow the same structure as the introduction and the abstract, but with more emphasis on the bottom part of the hourglass.

The summary should address the following points:

1. Restate the problem studied and explain why it is relevant.
2. Describe the main contribution and the approach taken (reference Chapter 3).
3. Name the central theoretical result (reference Theorem 3.4.1) and state informally what it guarantees.
4. Summarize the experimental findings – number of additional examples solved, runtime overhead – if an evaluation was conducted (reference Chapter 4).

Each future work item should:

1. Identify a specific limitation of the current approach or a natural extension that was beyond the scope of this thesis.
2. Describe what would be needed to address it (new theoretical machinery, a different encoding, a richer formalism, etc.).
3. Explain what benefit this would bring (more examples solved, a stronger guarantee, applicability to a wider class of systems).

Bibliography

- [1] Martin Avanzini, Ugo Dal Lago, and Akihisa Yamada. On probabilistic term rewriting. *Science of Computer Programming*, 185:102338, 2020.
- [2] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, Cambridge, UK, 1998.
- [3] Jan-Christoph Kassing and Jürgen Giesl. From innermost to full probabilistic term rewriting: Almost-sure termination, complexity, and modularity. *Logical Methods in Computer Science*, 21(4):28:1–28:79, 2025.
- [4] Jan-Christoph Kassing and Jürgen Giesl. The annotated dependency pair framework for almost-sure termination of probabilistic term rewriting. *Science of Computer Programming*, 251:103417, 2026.
- [5] Matthias Raffelsieper and Hans Zantema. A transformational approach to prove outermost termination automatically. *Electronic Notes in Theoretical Computer Science*, 237:3–21, 2009. Proceedings of the 8th International Workshop on Reduction Strategies in Rewriting and Programming (WRS 2008).
- [6] René Thiemann. From outermost termination to innermost termination. In *SOFSEM 2009: Theory and Practice of Computer Science*, volume 5404 of *Lecture Notes in Computer Science*, pages 533–545, Berlin, Heidelberg, 2009. Springer.